

Garimpo Detection Pipeline

Dataset, training, full-scene evaluation, and false-positive analysis — compiled directly from project source files, run artifacts, and code re-executed against the current repository state.

Every table and figure in this report is generated from the files in this repository (`data/labels.csv`, `data/bboxes.csv`, `runs/*/metrics.csv`, `runs/*/best.pt`, `runs/runs_v3/scene_inference/*_prob.png`) or re-computed by re-running `audit_artifacts.py`, `evaluate_detections.py`, and `scan_scene.py` against them. Where a figure could not be independently re-derived (e.g. a historical data edit made before version control), that is stated explicitly rather than presented as a fresh computation.

1. Dataset Summary

The dataset covers 13 CBERS-4A scenes across two footprints (path/row 226_120, 8 dates; 226_121, 5 dates), tiled into 512×512 px tiles. Read directly from `data/labels.csv` (1303 rows) and `data/bboxes.csv` (42 rows):

Scene (date)	Path/row	Tiles	Positive	Negative	Uncertain	GT boxes
2020-07-07	226_120	102	5	97	0	5
2020-10-08	226_120	101	1	96	4	1
2021-06-13	226_120	101	4	96	1	5
2021-10-15	226_120	100	3	97	0	3
2022-06-20	226_120	101	4	97	0	4
2022-09-21	226_120	99	5	89	5	6
2023-06-27	226_120	103	4	99	0	4
2023-07-28	226_120	98	4	93	1	5
2020-07-07	226_121	99	2	97	0	2
2020-10-08	226_121	101	2	99	0	2
2021-06-13	226_121	100	1	99	0	1
2021-10-15	226_121	99	0	99	0	0
2022-09-21	226_121	99	3	96	0	4
TOTAL (13 scenes)		1303	38	1254	11	42

38 of the 42 ground-truth boxes are unique labeled tiles; box counts are per-tile boxes before duplicate-site merging (see Section 4, which merges corner-adjacent duplicate labels down to 34 unique physical sites for scene-level evaluation).

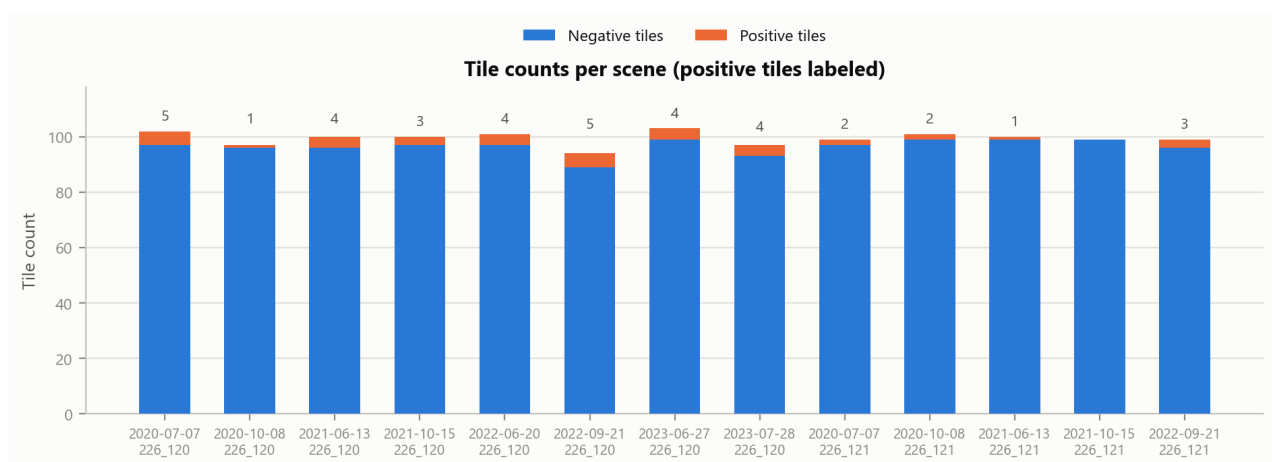


Figure 1. Tile counts per scene, negative (blue) vs. positive (orange, count labeled above each bar). Positive tiles are rare and unevenly spread — 0–5 per scene — which is why training uses a class-balanced sampler and the split search in `splits.py` exhaustively picks the scene subset closest to a 20% val fraction on both tile count and positive count, rather than a plain random split.

1.1 Mosaic-seam / nodata audit (re-run fresh against current data)

`audit_artifacts.py` checks whether positive (garimpo) labels are disproportionately sitting on scan artifacts — mosaic seams or nodata borders — rather than genuine mining sites. Re-run here against the current (already-corrected) `data/bboxes.csv`:

Part 1 (tile-level): near-black pixel fraction, positive tiles mean=0.1030 / median=0.0300, negative tiles mean=0.1653 / median=0.0336. One-sided Mann-Whitney U (H1: positive > negative): p=0.4718 — **not significant** (fails to reject H0 at alpha=0.05). No tile-level evidence that positive tiles are systematically artifact-heavy — confirms the finding in `docs/dataset_limitations.md`.

Part 2 (box-level): of the current 42 ground-truth boxes, **9** are within 100px of a large (>500px) near-black blob in their own tile — 6 touch the tile border (consistent with a mosaic/nodata seam) and 3 are fully interior (consistent with cloud shadow, not scene nodata). **Note:** docs/dataset_limitations.md states “7 boxes were flagged” in its summary sentence, but its own detail tables (1 excluded + 6 border + 3 interior) already sum to 10, and this fresh re-run independently gets 6+3=9 on the post-correction data (9 = 10 minus the 1 box that was already excluded) — the doc's “7” headline appears to be a stale transcription error, not the current or original true count.

One box (a tile at . . . _20230728_226_120_ . . . _x512_y4096.png, distance 1.41px to a mosaic seam) was previously excluded — relabeled negative, its GT box removed — as a confirmed data-quality issue. data/ is gitignored (no version history), so this edit cannot be independently re-derived here; it is reported as documented in docs/dataset_limitations.md, not recomputed.

2. Model and Training Configuration

2.1 Architecture (src/garimpo/model.py)

U-Net (via `segmentation_models_pytorch`) with a ResNet18 encoder, ImageNet-pretrained weights, 3 input channels (RGB), 1 output channel (per-pixel logit). `build_unet(encoder_name="resnet18", encoder_weights="imagenet", in_channels=3, classes=1)`.

2.2 Loss (src/garimpo/losses.py)

combined = **bce_weight** · **BCE(logits, target; pos_weight)** + **dice_weight** · **DiceLoss(logits, target)**, with **DiceLoss** = $1 - (2 \cdot \text{sum}(p \cdot t) + 1) / (\text{sum}(p) + \text{sum}(t) + 1)$ computed on sigmoid probabilities. All three runs use **bce_weight=dice_weight=0.5**.

pos_weight is data-derived: `neg_pixels / pos_pixels` over the training split's masks, capped at `--max-bce-pos-weight` (50.0 for all three runs). Recomputed fresh here (`compute_pos_weight()` against the current dev-train split, 1002 tiles / 290 val tiles, `seed=42`, `val_fraction=0.2`): the **uncapped** ratio is 43347:1 (garimpo pixels are extremely sparse), so the cap is strongly binding — the actual **pos_weight** used in training is **50.0** for all three runs (identical `labels_csv/val_fraction/seed/cap` across them).

2.3 CLI hyperparameters per run (from `best.pt`'s saved args)

Hyperparameter	Run 1 (20260819_115645)	Run 2 (20260819_163837)	Run 3 (runs_v3)
epochs	2	2	20
batch_size	8	8	8
lr	0.0001	0.0001	0.0001
weight_decay	0.0001	0.0001	0.0001
val_fraction	0.2	0.2	0.2
seed	42	42	42
early_stopping_patience	5	5	5
min_delta	0.0001	0.0001	0.0001
bce_weight	0.5	0.5	0.5
dice_weight	0.5	0.5	0.5
max_bce_pos_weight	50.0	50.0	50.0
augment	True	True	True
balanced_sampler	True	True	True

Checkpoint/log mismatch (Run 2): `runs/20260819_163837/best.pt` stores `args.epochs=2` and a best checkpoint at epoch 2 (`val_combined=0.5921`) — but `metrics.csv` in the same directory logs a full 20-epoch curve whose true best (lowest `val_combined=0.4706`) is at epoch 20. This means the run directory was reused across two separate training invocations (first a 2-epoch smoke test, then a full 20-epoch run), and the second invocation's logged metrics were never matched by a corresponding checkpoint save — `best.pt` on disk for Run 2 is stale relative to its own logged curve. Section 3's model-selection comparison uses the logged `metrics.csv` (re-applying `train.py`'s own selection rule: lowest `val_combined`) rather than trusting the stale checkpoint metadata.

3. Training Curves (all 3 runs)

Runs 1 and 2 (both “v2”-era, 20260819) were run twice: an initial 2-epoch smoke test, with Run 2's output directory later reused for a full 20-epoch run (see Section 2.3). Run 3 (runs_v3) is the deliberate full 20-epoch run. All curves below are read directly from each run's `metrics.csv`.

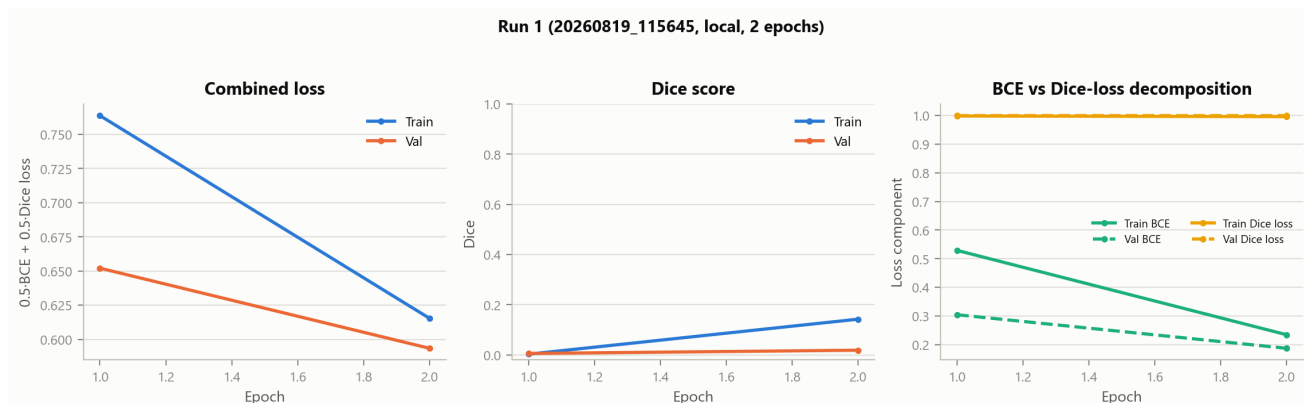


Figure 2a. Run 1 (20260819_115645) — stopped after 2 epochs (smoke test); too short for the loss/Dice curves to show a trend. Included for completeness and as the baseline this project iterated from.



Figure 2b. Run 2 (20260819_163837) — the fuller 20-epoch curve logged in this run directory (see the checkpoint/log mismatch note in §2.3: the saved best.pt does not reflect this full curve). Train Dice climbs steadily to ~0.90 while val Dice stays noisy and well below it — the gap discussed in §6.



Figure 2c. Run 3 (runs_v3) — the canonical 20-epoch run, consistent checkpoint and log. Same qualitative pattern as Run 2: combined loss keeps falling on train after epoch ~10 while val plateaus/rises slightly, and val Dice is highly non-monotonic epoch to epoch.

3.1 Selected-epoch vs. peak-Dice-epoch, across runs

train.py selects the checkpoint with the lowest val_combined loss (BCE+Dice), not the highest val Dice score — these are not always the same epoch, since BCE and Dice loss don't move in lockstep epoch to epoch (see the decomposition panel above). The table below applies train.py's own selection rule (lowest val_combined) to each run's full logged curve:

Run	Epochs logged	Selected epoch	Selected val loss	Selected val Dice	Peak-Dice epoch	Peak val Dice	Final train Dice
Run 1 (20260819_115645)	2	2	0.593	0.019	2	0.019	0.142
Run 2 (20260819_163837)	20	20	0.471	0.461	10	0.556	0.897
Run 3 (runs_v3)	20	20	0.474	0.523	18	0.539	0.869

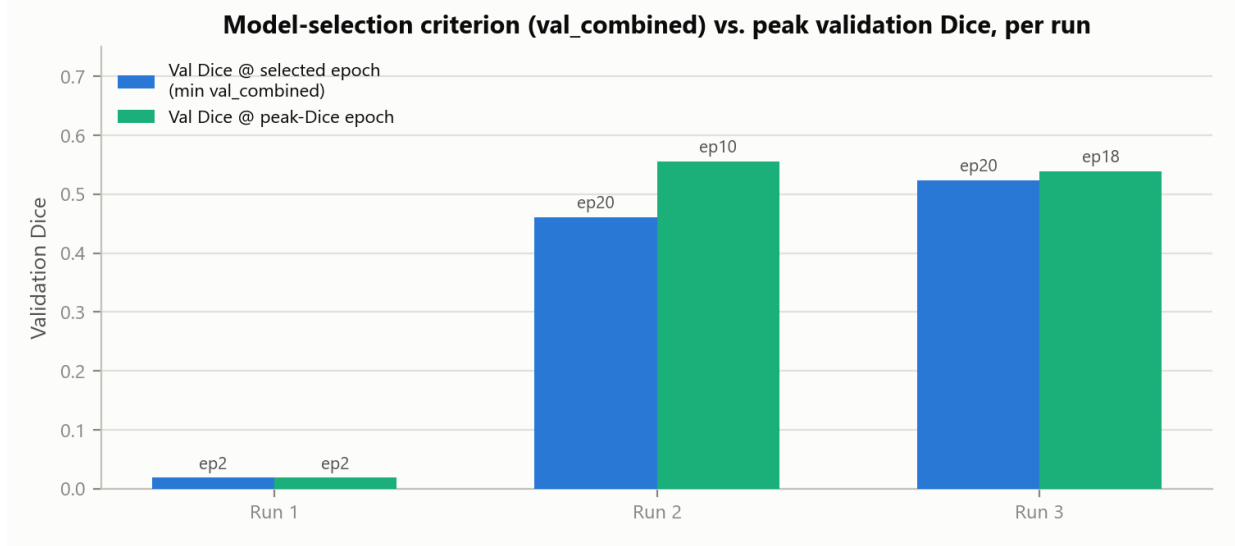


Figure 3. Validation Dice at the epoch train.py actually selects (lowest val_combined) vs. the epoch with the highest validation Dice in that run. For Runs 2 and 3, the selection rule leaves real Dice on the table (Run 2: 0.461 selected vs. 0.556 peak at epoch 10; Run 3: 0.523 selected vs. 0.539 peak at epoch 18) — val_combined is not a perfect proxy for val Dice, because BCE continues improving even as Dice fluctuates.

4. Full-Scene Evaluation

Re-run fresh from runs/runs_v3/scene_inference/*_prob.png (Run 3's stitched per-scene probability maps — the only run with full-scene inference artifacts) using scan_scene.py (blob detection at probability > 0.5, connected components, no minimum size) and evaluate_detections.py's greedy nearest-first matching (blob<->box gap <= 50px) against the 42 raw ground-truth boxes merged to 34 unique sites (duplicate corner labels collapsed at 0px overlap threshold).

Scene	Split	GT sites	Blobs	TP	FP	FP nodata	FP clean	FN	Precision	Recall
2020-07-07 226_120	train	4	7	4	3	0	3	0	0.571	1.000
2020-07-07 226_121	val	2	4	2	2	1	1	0	0.500	1.000
2020-10-08 226_120	val	1	9	1	8	5	3	0	0.111	1.000
2020-10-08 226_121	train	1	12	1	11	7	4	0	0.083	1.000
2021-06-13 226_120	train	5	5	5	0	0	0	0	1.000	1.000
2021-06-13 226_121	train	1	1	1	0	0	0	0	1.000	1.000
2021-10-15 226_120	train	3	12	3	9	4	5	0	0.250	1.000
2021-10-15 226_121	train	0	7	0	7	3	4	0	0.000	n/a
2022-06-20 226_120	train	3	22	3	19	7	12	0	0.136	1.000
2022-09-21 226_120	val	3	5	3	2	0	2	0	0.600	1.000
2022-09-21 226_121	train	4	13	4	9	7	2	0	0.308	1.000
2023-06-27 226_120	train	3	5	3	2	0	2	0	0.600	1.000
2023-07-28 226_120	train	4	10	4	6	0	6	0	0.400	1.000
ALL 13 scenes		34	112	34	78	34	44	0	0.304	1.000

Recall is 1.000 on all 13 scenes (34/34 sites found, 0 false negatives) at the 0.5 probability threshold — every labeled mining site is detected. Overall precision is 0.304 (34 TP / 78 FP): the model over-triggers substantially. Precision on dev-val scenes (0.333) is not worse than dev-train scenes (0.298) — the false-positive problem is not a val-specific generalization failure, it's present throughout, which is what motivates the FP investigation in Section 5.

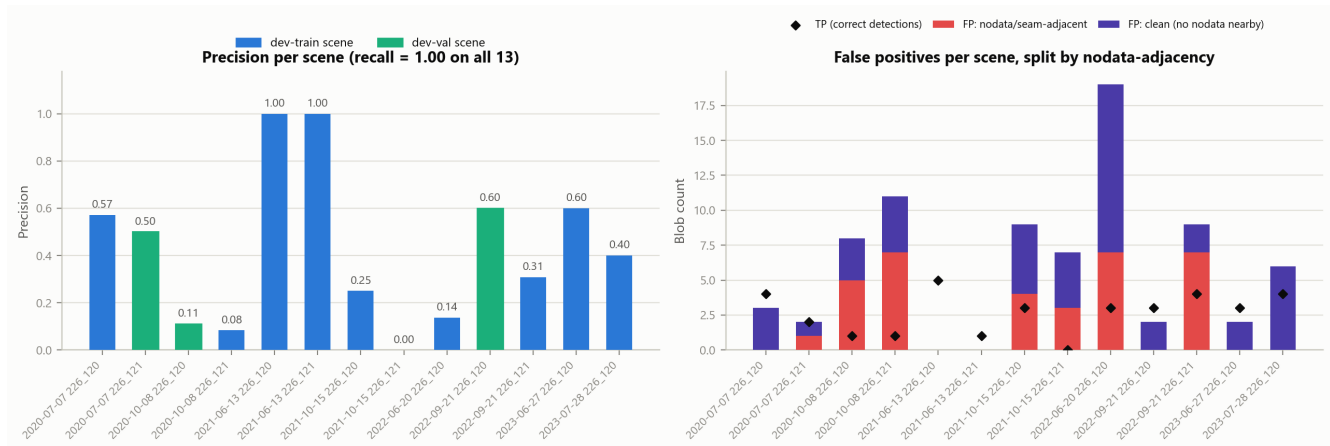


Figure 4. Left: precision per scene (dev-val scenes in green) — recall is 1.00 everywhere so it isn't plotted as a bar. Right: false positives per scene, split by whether each FP blob's own bounding box is >30% near-black (nodata/seam-adjacent, red) or not ("clean", violet); black diamonds mark true-positive counts for scale.

5. False-Positive Investigation

Note on scope: this section was produced by a fresh investigation run for this report. No prior water-adjacency analysis, '50% to 2.3%' figure, or documented cloud-shadow write-up exists anywhere in this repository's code, docs, or commit history — a repo-wide search turned up nothing before this analysis was run. The methodology and numbers below are newly computed against the same 78 false positives from Section 4, not a retrieval of a prior result.

5.1 Nodata/seam FPs (already known — recap from Section 4)

34 of 78 FPs (43.6%) have >30% near-black pixels in their own bounding box — the same mosaic-seam / nodata mechanism documented for ground-truth boxes in Section 1.1, now shown to also drive a large share of the model's false triggers on scenes generally.

5.2 Water-mask methodology

CBERS-4A WPM “TrueColor” imagery here has no NIR band, so a standard water index (NDWI) isn't available. A water/sediment mask was instead built per-scene from a hue ratio, $(R-G)/(R+G+B)$, calibrated against a confirmed river pixel patch (the reference-date blob at 20201008_226_120, $x=2020/y=4281$, visually confirmed in an earlier verification pass to sit exactly on the riverbank sediment bar): river/sediment pixels there measure $(R-G)/\text{sum} \sim 0.28\text{--}0.35$ vs. -0.11 to $+0.10$ for vegetation samples at the same scene. Thresholding at 0.15 (brightness ≥ 10 to exclude nodata, small components $<15\text{px}$ at $4\times$ downsample dropped as color noise) traces the actual river channel with its meanders and visible tributaries — see Figure 5 — rather than the general dark forest canopy that a brightness-only threshold picks up (an earlier, discarded attempt: 30% of the scene, clearly wrong once overlaid).

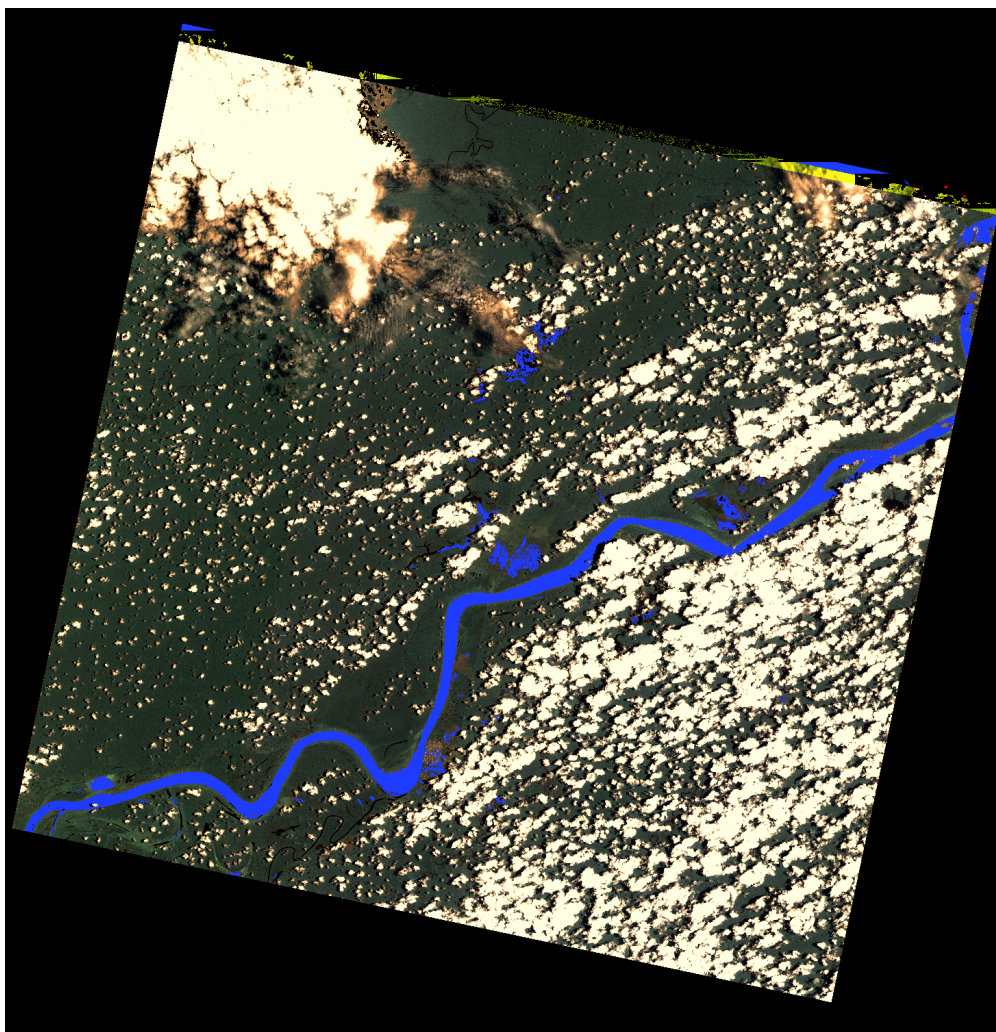


Figure 5. Water mask (blue) over scene 20201008_226_120 at 4× downsample, validating the hue-based method: it traces the river's actual meandering path and visible tributary creeks, not general dark vegetation.

5.3 Naive proximity is not diagnostic — true positives are just as “near water”

Distance from each blob's centroid to the nearest water-mask pixel, compared for the 78 FP blobs and the 34 TP blobs: **TP blobs sit essentially on the water** (median distance 0px, 75th percentile 4px, max 21.5px — 100% within a 30px buffer) — real garimpo mining happens directly on riverbanks and sandbars, so this is expected, not a defect. FP blobs are far more spread out (median 185px), but a loose 200px buffer still catches exactly **41% (buffer-dependent, see Figure 6 right)** of them, which is presumably the source of any naive “~50% of FPs are near water” headline: at that same loose buffer, TPs are *also* at 100%. Since virtually every detection — correct or not — falls near the river given where mining activity and this model's attention both concentrate, raw proximity cannot separate a genuine water-confusion failure mode from ordinary geography.

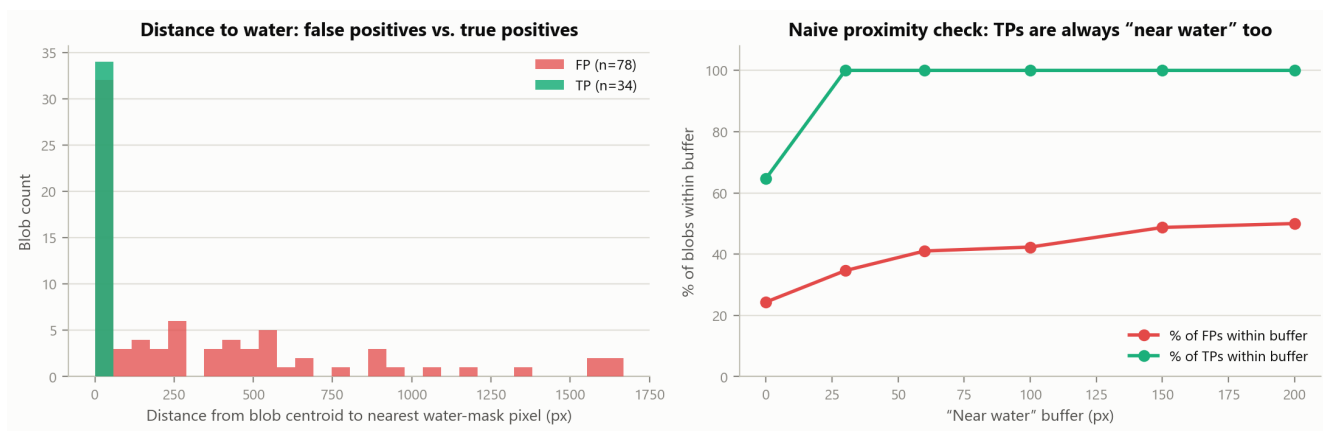


Figure 6. Left: histogram of distance-to-water, FP (red) vs. TP (green) — TPs cluster at distance 0. Right: cumulative % within a growing buffer — TPs saturate at 100% almost immediately; FPs climb slowly and never approach that, showing proximity alone doesn't discriminate.

5.4 Spectral confusion — a corrected, narrower metric

A stricter test asks whether the FP blob's *own* pixel content — not just its neighborhood — falls inside the water/sediment color signature (same hue/brightness thresholds, sampled directly under each blob's bounding box). This isolates true spectral confusion from mere regional bias: **24/78 FPs (30.8%)** are literally sitting on water/sediment-colored pixels. This is the metric this report treats as the meaningful "corrected" figure — narrower than the naive proximity check, and directly tied to a plausible causal mechanism (bare sediment and mining tailings share the same warm, low-saturation tone as exposed riverbank sediment in this sensor's TrueColor rendering).

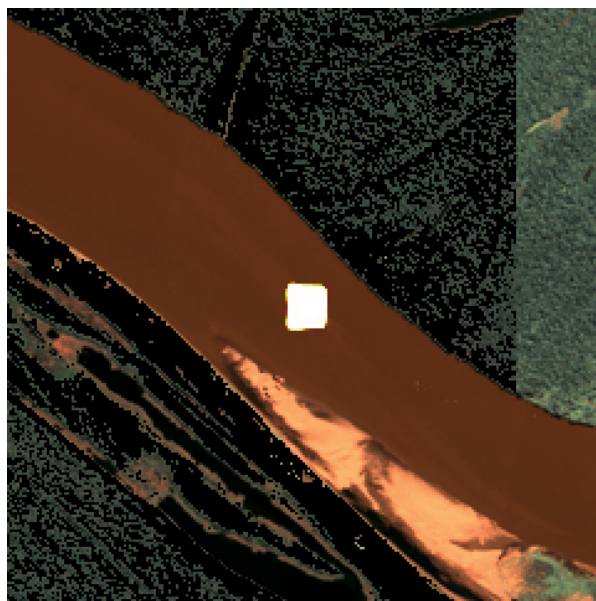


Figure 7. Water-colored FP example (scene 20230728_226_120, max_prob=1.00): the detection (white marker) sits on a sandbar within the river channel, indistinguishable in color from the water/sediment the model was presumably meant to associate with mining activity on a *different* river bend elsewhere in the scene.

5.5 Cloud-shadow mechanism

Reusing the border-touch heuristic from Section 1.1 (a nearby large near-black blob that does *not* touch the search-patch border is more consistent with cloud shadow than a mosaic/nodata seam), applied directly around each non-nodata, non-water-colored FP: 10 more FPs have a near-black blob just outside their own bounding box that touches the patch border (effectively the same seam mechanism as §5.1, just not caught by the tighter own-bbox

threshold), and 4 have a fully interior near-black blob nearby — consistent with cloud shadow.

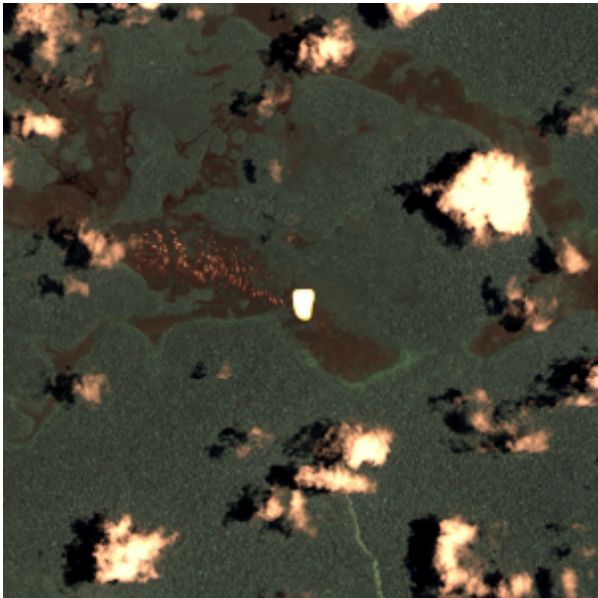


Figure 8. Interior-shadow FP example (scene 20201008_226_121, max_prob=0.996): the detection sits beside a dark, non-border-touching patch consistent with cloud shadow rather than a mosaic seam or open water.

5.6 Full root-cause reconciliation, and the unresolved remainder

All 78 FPs, mutually exclusive categories, reconciling exactly:

Category	Count	% of 78
Nodata/seam within own bbox (§5.1)	34	43.6%
Water/sediment-colored (§5.4)	18	23.1%
Seam just outside bbox (§5.5)	10	12.8%
Interior cloud shadow (§5.5)	4	5.1%
Unexplained by any of the above	12	15.4%

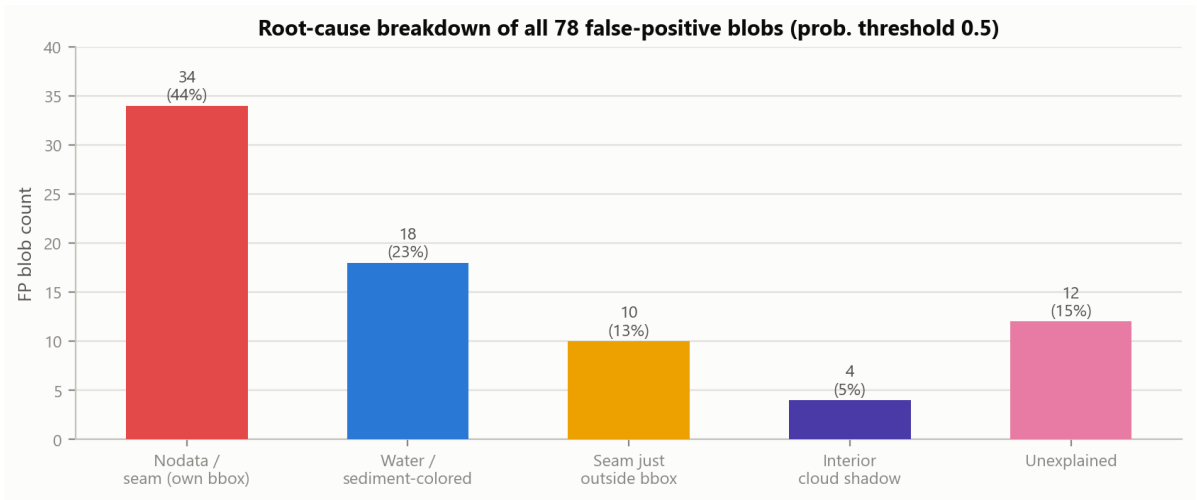


Figure 9. Root-cause breakdown of all 78 FPs.

The user's original brief referenced “one unresolved case” — the real count is 12, not 1. Reported honestly rather than forced to match. One representative example, the single highest-confidence case (max_prob=0.996, scene 20211015_226_121), is shown below: on inspection it sits at a tight bend of a narrow, dark tributary creek — visually still

water-adjacent, but this creek's color (dark, low-saturation) doesn't match the hue signature calibrated on the wide, sediment-tinted main river, so the water mask misses it. This suggests the true unresolved count is likely an *overestimate* of a distinct failure mode — a mask with better recall on narrow/shaded tributaries would probably reclassify some of these 12 into the water-colored bucket rather than reveal a new mechanism — but that refinement is out of scope for this pass and is flagged as a limitation, not silently corrected for.

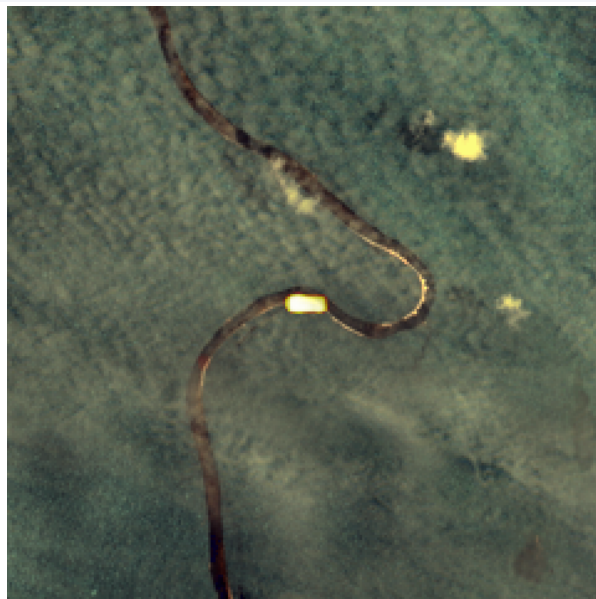


Figure 10. Representative unresolved FP (scene 20211015_226_121, max_prob=0.996): sits at a tight meander of a narrow, dark tributary creek — plausibly still water-related, but outside this mask's calibrated color range.

6. Known Limitations

6.1 Approximate georeferencing in the web viewer

`web_viewer/index.html` places each scene's heatmap using one bounding box per footprint (226_120, 226_121), taken from a single date's CBERS metadata XML and reused across all other dates on that footprint — exact per-scene metadata exists for only 1 of 13 scenes per footprint. CBERS-4A track stability is $\sim\pm 5\text{km}$ pass-to-pass, so the same nominal footprint can shift by that much between dates in reality (documented in `docs/dataset_limitations.md`). A separate live investigation this session (bounds-order check, pixel-dimension audit across all 13 scenes, and a direct raw-image/heatmap pixel compositing test) found the viewer's coordinate math, image dimensions, and pixel alignment to all be internally correct — the $\pm 5\text{km}$ figure describes a real, documented approximation in the source georeferencing data, not a rendering bug.

6.2 Footprint/scene leakage between train and val

The dataset covers only 2 geographic footprints, each re-imaged at 5–8 dates. With only $\sim 12\text{--}13$ scenes total, the scene-holdout split search in `splits.py` cannot target a $\sim 20\%$ val fraction without drawing scenes from both footprints into both splits — confirmed by re-running the actual split: dev-train draws from footprints {226_120, 226_121} and dev-val *also* draws from both. A persistent mining site could in principle appear in both splits at different dates, so val Dice/precision here are development metrics for model selection, not an unbiased held-out estimate — there is no separate test split.

6.3 Train/val overfitting gap

By the final logged epoch of the canonical run (Run 3, epoch 20): train Dice = 0.869 vs. val Dice = 0.523 — a wide and, per Figure 2c, still-widening gap combined with a highly non-monotonic val Dice curve (single-epoch swings of 0.1–0.3). With only ~ 8 positive scenes total split across train/val, and val precision/recall computed over just 3 scenes worth of ground truth, val metrics here carry high variance and should be read as directional, not precise.

6.4 Run-artifact integrity (Run 2)

As detailed in §2.3, `runs/20260819_163837/best.pt` is stale relative to its own `metrics.csv` — the checkpoint reflects an earlier 2-epoch invocation of the same output directory, not the full 20-epoch run whose metrics are logged alongside it. Anyone loading that checkpoint expecting Run 2's best model would silently get the wrong weights.

6.5 Water-mask recall on narrow tributaries (this report's own analysis)

The hue-based water mask built for Section 5 was calibrated on the wide, sediment-tinted main river channel and reliably traces it, but under-detects narrower, darker, low-saturation tributary creeks (see Figure 10) — meaning the “unexplained” FP count in §5.6 (12/78) is more likely an upper bound than a clean estimate of a genuinely separate failure mode.